

A PRELIMINARY PROJECT REPORT ON

**AUTOMATED VIDEO CONTENT
MANAGEMENT SYSTEM WITH
COPYRIGHT ENFORCEMENT**

**SUBMITTED TO THE SAVITRIBAI PHULE UNIVERSITY, PUNE
IN THE PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE**

OF

BACHELOR OF ENGINEERING (Computer Engineering)

SUBMITTED BY

**DURGESH CHOUDHARY
SIDDHARTH HUMBE
PIYUSH LAVEKAR
GANESH LOKHANDE**

**Exam No: B400050359
Exam No: B400050413
Exam No: B400050473
Exam No: B400050478**



**DEPARTMENT OF COMPUTER ENGINEERING
Pune Institute of Computer Technology
Dhankawadi, Pune - 411043**

**SAVITRIBAI PHULE PUNE UNIVERSITY
2024-2025**



CERTIFICATE

This is to certify that the project report entitled
Automated Video Content Management System with Copyright Enforcement

Submitted by

DURGESH CHOUDHARY
SIDDHARTH HUMBE
PIYUSH LAVEKAR
GANESH LOKHANDE

Exam No: B400050359
Exam No: B400050413
Exam No: B400050473
Exam No: B400050478

is a bonafide student of this institute and the work has been carried out by him/her under the supervision of **Prof. Nikita Kapadnis** and it is approved for the partial fulfillment of the requirement of Savitribai Phule Pune University, for the award of the degree of **Bachelor of Engineering** (Computer Engineering).

Prof. Nikita Kapadnis
Internal Guide
Dept. of Computer Engg.

Dr. G. V. Kale
Head
Dept. of Computer Engg.

Dr. S. T. Gandhe
Principal
Pune Institute of Computer Technology

Place: Pune
Date: 30/04/2025

ACKNOWLEDGEMENT

*It gives us great pleasure in presenting the project report on ‘**Automated Video Content Management System with Copyright Enforcement**’.*

*We would like to thank our project guide **Prof. Nikita Kapadnis** for giving us all the help and guidance we needed. We are really grateful to them for their kind support and valuable suggestions.*

*We are also grateful to **Dr G. V. Kale**, Head of Computer Engineering Department, PICT for her indispensable support and suggestions throughout the project. We also express our gratitude to **Dr. A. R. Deshpande**, BE project coordinator, for her timely guidance and support.*

B400050359 DURGESH CHOUDHARY

B400050413 SIDDHARTH HUMBE

B400050473 PIYUSH LAVEKAR

B400050478 GANESH LOKHANDE

(B.E. Computer Engg.)

ABSTRACT

This report explores the design and implementation of a web application for video content management, with a specific focus on identifying duplicate uploads and managing copyright strikes. In the proposed system, users can upload videos, while the backend generates unique fingerprints for each video using content-based retrieval techniques. The fingerprinting process includes frame normalization, segmentation, and resizing, enabling efficient comparison against an existing fingerprint database to detect potential duplicates. When a duplicate is found, the system automatically issues a copyright strike, notifying the user and logging the violation in their profile.

Key features such as user registration, video metadata management, and content moderation ensure an optimal user experience while upholding copyright protection standards. Additionally, the platform incorporates an appeals process for users to contest strikes, and a robust admin dashboard for overseeing content, strikes, and user activity. The report further discusses the underlying technology stack, including cloud storage for scalability, video processing algorithms, and performance optimization strategies. This system offers a scalable solution for managing user-generated video content, maintaining copyright integrity, and automating moderation processes.

Contents

1	Introduction	1
1.1	Overview	2
1.2	Motivation	2
1.3	Problem Definition and Objectives	2
1.4	Project Scope & Limitations	3
1.5	Methodology of Problem solving	4
2	Literature Survey	5
2.1	Literature survey	6
3	Software Requirements Specification	9
3.1	Assumptions and Dependencies	10
3.1.1	Assumptions	10
3.1.2	Dependencies	10
3.2	Functional Requirements	12
3.2.1	System Feature 1: Video Upload and Processing	12
3.2.2	System Feature 2: TIRI-Based Fingerprint Generation	12
3.2.3	System Feature 3: Copyright Detection	13
3.2.4	System Feature 4: User Interface and Results Display .	13
3.3	External Interface Requirements	13
3.3.1	Hardware Interfaces	13
3.3.2	Software Interfaces	14
3.3.3	Other Requirements	14
3.3.4	Safety Requirements	14
3.4	Nonfunctional Requirements	15
3.4.1	Performance Requirements	15
3.4.2	Safety Requirements	15
3.4.3	Security Requirements	15
3.5	System Requirements	16
3.5.1	Database Requirements	16
3.5.2	Software Requirements	16

3.5.3	Hardware Requirements	16
3.6	Analysis Models: SDLC Model to Be Applied	17
3.6.1	Agile Model Application	17
4	System Design	18
4.1	Project Architecture	19
4.2	Data Flow Diagram	20
4.3	Class Diagram	21
4.4	Use Case Diagram	22
4.5	Sequence Diagram	23
5	Project Plan	24
5.1	Project Estimate	25
5.1.1	Reconciled Estimates	25
5.1.2	Project Resources	25
5.2	Risk Management	27
5.2.1	Risk Identification	27
5.2.2	Risk Analysis	27
5.2.3	Overview of Risk Mitigation, Monitoring, Management	27
5.3	Project Schedule	29
5.3.1	Project Task Set	29
5.3.2	Timeline Chart	30
5.4	Team Organization	30
5.4.1	Team Structure	30
6	Project Implementation	31
6.1	Stages of Implementation	32
6.1.1	TIRI-Based Fingerprinting Baseline	32
6.1.2	Modular Python Service	32
6.1.3	Integration with Cloud-Based Storage	32
6.1.4	Final Pipeline Assembly	32
6.1.5	Summary	33
6.2	Project Modules	33
6.2.1	Overview of Project Modules	33
6.2.2	Tools and Technologies Used	34
6.2.3	Algorithm Details	34
7	Software Testing	35
7.1	Types of Testing	36
7.1.1	Alpha Testing	36
7.1.2	Beta Testing	36

7.2	Integration Testing	37
7.2.1	Objectives	37
7.2.2	Approach	37
7.2.3	Results	38
8	Results	39
8.1	Output	40
8.2	Screenshots	42
9	Conclusions	45
9.1	Summary	46
9.2	Applications	46
9.3	Limitations	47
9.4	Future Scope	47
9.5	Conclusion	48
10	Appendix	49
10.1	Appendix A	50
10.1.1	Satisfiability Analysis	50
10.1.2	Complexity Class Analysis	50
10.1.3	Modern Algebra Perspective	51
10.2	Appendix B	52
10.2.1	Survey Paper Details	52
10.2.2	Implementation Paper Details	53
10.3	Appendix C	54
10.3.1	Plagiarism Report	54
11	References	56

List of Figures

4.1	Project Architecture	19
4.2	Data Flow Diagram	20
4.3	Entity - Relationship Diagram	21
4.4	Use Case Diagram	22
4.5	Sequence Diagram	23
5.1	Summarized Timeline Chart	30
8.1	Application Interface	42
8.2	Video Uploading stage	42
8.3	Dashboard with uploaded video	43
8.4	Copyright Strike and video removed	43
8.5	backend report	44
10.1	IEEE Xplore Snapshot	52
10.2	Authors	52
10.3	Conference Details	53
10.4	Overall Similarity Report	54
10.5	Document Details	55

CHAPTER 1

INTRODUCTION

1.1 Overview

The project focuses on developing a video sharing platform with an integrated copyright protection system that automatically detects duplicate content and manages copyright strikes. The platform enables users to upload videos while implementing sophisticated fingerprinting techniques to identify potential copyright violations. The system features user-friendly interfaces for content management, automated content and a comprehensive strike management system. Key features include video fingerprinting algorithms that create unique signatures for uploaded content, real-time duplicate detection, and a fair appeals process for handling copyright disputes. The project emphasizes maintaining platform integrity while ensuring a seamless user experience, incorporating cloud storage solutions and content delivery networks for optimal performance. The platform aims to deliver a balanced environment that protects content creators' rights while fostering a vibrant community of video creators and viewers.

1.2 Motivation

The digital content creation industry is experiencing unprecedented growth in market . This gap presents a significant opportunity for developing sophisticated video sharing platforms with built-in copyright protection mechanisms. The increasing instances of content piracy and unauthorized redistribution highlight the urgent need for automated systems that can detect and prevent copyright violations. By focusing on developing a video platform with advanced copyright protection features, this project aims to contribute to the digital content ecosystem, fostering a safer environment for content creators while promoting innovation in content protection technologies.

1.3 Problem Definition and Objectives

The project titled "Development of Video Sharing Platform with Copyright Strike System" addresses the challenge of creating a scalable video sharing platform that effectively protects content creators' rights through automated duplicate detection and copyright strike management. As digital content creation continues to grow exponentially, it is crucial to develop platforms that not only facilitate content sharing but also ensure proper attribution and protection of intellectual property. The complexity of video fingerprinting and the need for real-time processing often leads to challenges in accurately

identifying duplicate content while maintaining system performance. Thus, there is a pressing need for a platform design that balances efficient content delivery with robust copyright protection mechanisms.

The primary objective of this project is to develop a video sharing platform that effectively implements features such as video fingerprinting, automated duplicate detection, and strike management, all while maintaining high performance and user satisfaction. The project aims to deploy the platform in a production environment, allowing content creators to share their work confidently with built-in copyright protection. By achieving these objectives, the project will contribute to the growing landscape of digital content protection in India and set a standard for future content sharing platforms.

1.4 Project Scope & Limitations

The development of the platform faces several limitations and challenges. One significant challenge is the processing power required for video fingerprinting and real-time comparison of uploaded content. Ensuring quick response times while maintaining accuracy in duplicate detection requires careful optimization of system resources.

Additionally, storage and bandwidth limitations pose significant challenges, especially for a platform handling large video files. The need for efficient content delivery networks (CDN) and scalable storage solutions must be balanced with cost considerations and system performance.

Moreover, the complexity of video fingerprinting algorithms presents another constraint. While existing algorithms can detect exact matches, identifying partial matches or deliberately modified content requires more sophisticated approaches. The system must also handle false positives and negatives appropriately to maintain user trust.

In summary, while this project seeks to create an efficient video sharing platform with robust copyright protection, it must navigate challenges related to processing power, storage limitations, and algorithm accuracy. Addressing these challenges is crucial for the successful implementation and deployment of the platform.

1.5 Methodology of Problem solving

The methodology for developing the video sharing platform will involve a systematic approach to ensure effective implementation and scalability. We plan to utilize modern web technologies such as Node.js for the backend and React for the frontend, leveraging their robust capabilities for creating scalable web applications. The project will begin with the design of system architecture, database schemas, and API endpoints, followed by the implementation of core features such as video upload, processing, and storage. Development will incorporate essential elements such as video fingerprinting algorithms, user authentication, and strike management systems. We will utilize cloud services like Cloudinary for video storage, enabling efficient video streaming and storage management. Throughout the development process, we will implement various algorithms related to video processing, such as pHash for generating video fingerprints, content-based retrieval algorithms for duplicate detection, and automated moderation systems using AI tools. Regular testing and optimization will be conducted to identify and resolve any performance bottlenecks or technical issues. Feedback from user testing sessions will be crucial in refining the platform features and enhancing user experience. By following this comprehensive methodology, we aim to create a robust, scalable video sharing platform that effectively protects content creators' rights while providing an excellent user experience.

CHAPTER 2

LITERATURE SURVEY

2.1 Literature survey

In recent years, video hashing and fingerprinting have become essential for content authentication, copyright protection, duplicate detection, and retrieval in multimedia systems. Various approaches have been proposed to enhance robustness against distortions such as compression, noise, and temporal manipulations. Khelifi and Bouridane (2019) introduced a dual-purpose hashing technique robust to tampering and distortions. Kim et al. (2023) focused on adversarial training to improve deep neural networks under compression. Esmaili and Ward (2010) and Esmaili et al. (2011) developed TIRI-based methods to capture spatial-temporal features for copy detection and hashing. Chao (2008) and Sarkar et al. (2008) analyzed global and local descriptors like CFMT and SIFT for video fingerprinting. Aguilera et al. (2021) employed motion and semantic cues for forgery detection, while Singh et al. (2020) surveyed hybrid CBVR systems using traditional and deep learning features. Lastly, Liu et al. (2012) proposed GOS and FE-based descriptors for scalable, efficient near-duplicate detection.

A study by Khelifi and Bouridane (2019) [1] introduced a perceptual video hashing system designed for both content identification and authentication using a single hash. The system uses a novel DCT/DST-based signal calibration technique to extract hashes robust to common signal processing and sensitive to tampering. It implements distinct similarity measures tailored for identification and authentication tasks. Experimental results showed superior performance over state-of-the-art methods in handling distortions and detecting forgeries, while also allowing localization of tampered regions.

A framework by Kim et al. (2023) [2] proposed an adversarial compression-aware training scheme for enhancing the robustness of deep neural networks against video compression artifacts. The study models video encoders as differentiable modules within an end-to-end pipeline, enabling gradient-based optimization during training. Extensive experiments demonstrated that the proposed method improved classification accuracy under various compression levels. The framework outperformed conventional augmentation and compression-agnostic models on benchmark video datasets.

Esmaili and Ward (2010)[3] proposed robust video hashing algorithms based on Temporally Informative Representative Images (TIRIs) for video copyright protection. Their methods incorporate both spatial and temporal

information to generate compact and robust binary hashes. They compared their TIRI-based approaches against a 3D-DCT hashing baseline and showed higher robustness to noise, frame dropping, and time shifts. Experiments on a video dataset revealed superior true positive rates and precision, with the new algorithm being computationally more efficient.

Esmaeili et al. (2011) [4] proposed a robust and fast video copy detection system using content-based fingerprinting based on Temporally Informative Representative Images (TIRIs). Their TIRI-DCT method captures both spatial and temporal features, generating compact binary fingerprints. To enable fast matching in large databases, they introduced two approximate search techniques—inverted-file-based and cluster-based—favoring the latter for its speed and accuracy. The system demonstrated high detection performance across various attacks, and a secure version was developed to resist adversarial tampering.

Esmaeili and Ward (2010) [5] introduced a robust video hashing framework leveraging Temporally Informative Representative Images (TIRIs) combined with DCT to generate compact binary hashes. The method captures both spatial and temporal features by applying 2D-DCT to blocks of TIRIs, ensuring robustness against common video distortions. Experimental results on benchmark datasets showed improved precision and robustness compared to prior 3D-DCT-based methods. The approach demonstrated resilience to attacks such as noise, compression, and time shifts while maintaining computational efficiency.

Chao (2008)[6] presented a comprehensive classification of video fingerprinting techniques based on feature extraction methods, dividing them into global and local descriptor-based signatures. Global descriptors included color histograms, motion direction, ordinal measures, and transform-based methods like CFMT, while local descriptors covered techniques such as SIFT and ViCopT. The report emphasized robustness against transformations like scaling, rotation, compression, and frame drops. Experimental implementations focused on combining multiple features to improve accuracy and resilience.

Sarkar et al. (2008)[7] evaluated video fingerprinting techniques for detecting duplicate and similar videos using keyframe-based features like CFMT, SIFT, YCbCr histogram, and ordinal features. Their study applied image processing and frame-dropping attacks to generate test duplicates, showing that CFMT outperforms others for duplicate detection, while SIFT

excels in identifying similar videos. They also explored short clip queries against full-length videos, demonstrating CFMT's superiority in retrieval accuracy and compactness. Precision-recall analysis confirmed the trade-off between feature size and robustness across diverse distortions.

Aguileta et al. (2021)[8] proposed a video tampering detection system based on motion continuity using optical flow and semantic segmentation. The system identifies spatial and temporal inconsistencies by comparing motion vectors and object classes across frames. It employs a decision strategy that fuses motion, semantic, and temporal cues to pinpoint tampered regions. Experimental results showed high accuracy in detecting forgeries, particularly splicing and frame insertion, while maintaining low false positive rates.

Singh et al. (2020) [9] presented a comprehensive survey of content-based video retrieval (CBVR) systems, discussing key methods like shot boundary detection, keyframe extraction, and feature-based indexing using color, texture, motion, and deep learning features. The paper highlighted the shift towards machine learning and deep neural networks for improved semantic understanding. It also identified challenges such as scalability, semantic gap, and real-time retrieval efficiency. Trends pointed to hybrid models combining traditional and learning-based techniques for better performance across large-scale datasets.

Liu et al. (2012) [10] proposed a near-duplicate video detection framework using a compact local image descriptor called Gradient Ordinal Signature (GOS) and a Fixed-point Embedding (FE) similarity search method. GOS offers low-dimensional, rotation- and mirror-invariant features suitable for large-scale video analysis. FE efficiently maps these descriptors into indexable forms with high recall and low memory overhead, outperforming locality-sensitive hashing in speed and memory use. Experiments on benchmark datasets demonstrated the system's superior efficiency and robustness compared to state-of-the-art methods.

CHAPTER 3
SOFTWARE REQUIREMENTS
SPECIFICATION

3.1 Assumptions and Dependencies

3.1.1 Assumptions

- **Clean and Supported Video Input:** It is assumed that all uploaded videos are in widely supported formats (e.g., MP4, AVI) and are not corrupted or excessively compressed.
- **Consistent Fingerprint Generation Using TIRI:** The system assumes that the TIRI (Temporal Informative Representative Images)-based perceptual hashing algorithm provides robust and consistent fingerprints across slightly altered versions of the same video (e.g., changes in resolution, bitrate, or minor edits).
- **Unique Hashes for Distinct Content:** It is assumed that different videos will result in significantly different TIRI-based hashes, ensuring accurate copyright detection and minimizing false positives.
- **User Latency Tolerance:** It is assumed that users accept a short delay after video upload, during which the system processes the video, extracts TIRI frames, computes hashes, and performs fingerprint matching.
- **Prototype Data Scale:** In the prototype phase, the video dataset size is assumed to be moderate, allowing direct fingerprint comparisons without requiring advanced indexing or parallel processing.

3.1.2 Dependencies

Tech Stack

Frontend:

- **React.js** – For building the user interface with video upload, processing status, and results display.

Backend:

- **Node.js (Express)** – Handles API routing, file uploads, communication with Python service, and MongoDB integration.

Python Server:

- Handles the core video processing, including:

- Frame extraction
- TIRI generation
- Perceptual hashing
- Fingerprint comparison

Python Libraries

- *opencv-python* – For video decoding and frame extraction.
- *ffmpeg-python* – For additional video processing or format handling.
- *imagehash*, *PIL* – For generating perceptual hashes from frames.
- *scipy* – For applying Gaussian windows and DCT (Discrete Cosine Transform) and signal filtering in TIRI computation.
- *numpy* – For numerical processing, matrix manipulation, and efficient array operations.

Node.js Libraries

- *cloudinary* – For handling video file uploads.
- *axios* – For making HTTP requests from Node.js to the Python back-end.
- *mongoose* – For defining schemas and interacting with MongoDB.

Database

MongoDB: Stores metadata, TIRI fingerprints, and detection results. Collections may include:

- *videos* – Stores video details and file references.
- *fingerprints* – Stores generated perceptual hashes.
- *detection_logs* – Records copyright check results.

Environment and Infrastructure

- Moderate compute environment capable of handling video processing (CPU is sufficient, GPU optional).
- Local storage or Cloudinary for storing uploaded video files

3.2 Functional Requirements

This section outlines the functional requirements of the copyright detection system, structured by key system features.

3.2.1 System Feature 1: Video Upload and Processing

Description: Allows users to upload video files to the system for copyright analysis.

Functionalities:

- Accepts video files in common formats (e.g., MP4, AVI).
- Performs format validation and rejects corrupted or unsupported files.
- Transfers the uploaded file from the frontend (React) to the backend (Node.js) for processing.

Inputs: Video file (uploaded via browser).

Outputs: Upload status and acknowledgment of file receipt.

3.2.2 System Feature 2: TIRI-Based Fingerprint Generation

Description: Generates perceptual fingerprints using TIRI-based hashing for the uploaded video.

Functionalities:

- Extracts frames using OpenCV.
- Computes Temporal Informative Representative Images (TIRI).
- Applies Gaussian filtering, DCT, and perceptual hashing using SciPy and NumPy.
- Stores generated fingerprints in the MongoDB database.

Inputs: Video file path.

Outputs: Fingerprint data stored in the database.

3.2.3 System Feature 3: Copyright Detection

Description: Performs fingerprint comparison against existing records to detect possible copyright violations.

Functionalities:

- Compares perceptual hashes of uploaded video with stored hashes.
- Identifies matches based on predefined similarity thresholds.
- Logs results, including match confidence and source video metadata.

Inputs: Fingerprint of uploaded video.

Outputs: Detection results (match or no match), confidence score.

3.2.4 System Feature 4: User Interface and Results Display

Description: Provides a clean and intuitive interface for users to interact with the system and view detection results.

Functionalities:

- Allows users to upload videos and receive real-time status updates.
- Displays detection results clearly, including whether a match was found.
- Shows additional info such as similarity score, matched video title, and match timestamp.

Inputs: User interactions (upload button, form input).

Outputs: Visual display of detection status and results.

3.3 External Interface Requirements

3.3.1 Hardware Interfaces

- **Development:**
 - Mouse, keyboard, and high-resolution monitor for development environment interaction.
 - Adequate processing power and RAM for running development tools.

- **Production:**

- Compatible with standard web browsers (Chrome, Firefox, Safari).
- **PC/Mobile Requirements:**
 - * Minimum 4GB RAM (8GB recommended).
 - * Modern browser with JavaScript enabled and stable internet connection.

3.3.2 Software Interfaces

- **Frontend:** React.js for UI development.
- **Backend:** Node.js (Express.js) for backend API.
- **Database:** MongoDB for storing video metadata and fingerprints.
- **Authentication:** Token based for secure user authentication.
- **File Storage:** Cloudinary for video storage.
- **Python Service:** Flask to handle video fingerprinting and copyright detection logic.

3.3.3 Other Requirements

- **Testing:** Postman for API testing.
- **Collaboration Tools:** WhatsApp group and Google Meet for communication and meetings.

3.3.4 Safety Requirements

- **Data Privacy:** Secure encryption and data handling protocols for all user data.
- **User Consent:** Consent is required before processing video files and generating fingerprints.
- **Role-Based Access Control:** Ensures only authorized users access specific system functionalities.
- **Audit Logging:** Tracks all significant actions for compliance and security monitoring.

3.4 Nonfunctional Requirements

3.4.1 Performance Requirements

1. **Response Time:** Copyright detection results should be provided within a few seconds after video processing.
2. **Accuracy:** The system must minimize false positives and negatives in copyright detection.
3. **Scalability:** The system should scale efficiently as user demand and video uploads increase.
4. **Resource Efficiency:** The system should optimize CPU and memory usage, especially during peak traffic.

3.4.2 Safety Requirements

- **Data Protection:** Sensitive data must be encrypted and stored securely.
- **Compliance:** The system must comply with data protection regulations such as GDPR and CCPA.
- **System Reliability:** High availability with minimal downtime, and periodic data backups.

3.4.3 Security Requirements

- **Secure Communication:** All data transmission must be over HTTPS.
- **User Authentication:** Secure authentication via tokens.
- **Access Control:** Enforce role-based access control to protect sensitive operations.
- **API Security:** Implement security measures against common web vulnerabilities.

3.5 System Requirements

3.5.1 Database Requirements

- **Database:** MongoDB for storing video metadata, file references, and fingerprints.
- **Indexing:** Proper indexing of fingerprint data for efficient querying.

3.5.2 Software Requirements

- **Operating Systems:** Compatible with Windows, macOS, and Linux distributions.
- **Frontend Technologies:** React.js, HTML, CSS for the user interface.
- **Backend Technologies:** Node.js (Express.js) for backend development and Flask for the Python microservice.
- **Libraries:** Tokens for authentication, Cloudinary for video storage, Flask for the fingerprinting API, and MongoDB for database management.
- **Development Environment:** VS Code with Git for version control, Postman for API testing.

3.5.3 Hardware Requirements

- **Processor:** Multi-core processor with at least 2.0 GHz for optimal performance.
- **Storage:** Minimum 50 GB of free disk space for storing video data and fingerprints.
- **RAM:** At least 8 GB of RAM for smooth operation, especially for video processing.
- **Graphics Card:** Recommended for faster fingerprint generation and processing of large video files.

3.6 Analysis Models: SDLC Model to Be Applied

For this project, we followed the Agile SDLC model, which emphasizes iterative development, flexibility, and continuous feedback. Given the complexity of video fingerprinting and the need for frequent updates, Agile allowed us to break the project into manageable sprints, enabling continuous testing and refinement.

3.6.1 Agile Model Application

- **Requirement Gathering:** Identified features like video upload, fingerprinting, and copyright detection.
- **Design:** Created architecture diagrams and a modular design for microservices.
- **Implementation:** Developed modules using React, Node.js, Python, MongoDB, and Cloudinary.
- **Testing:** Performed alpha and beta testing to ensure functionality and accuracy.
- **Deployment:** Used cloud services and CI/CD tools for streamlined release.
- **Feedback & Improvement:** Incorporated user feedback to enhance performance and usability.

Agile ensured a responsive, scalable, and user-focused development process.

CHAPTER 4

SYSTEM DESIGN

4.1 Project Architecture

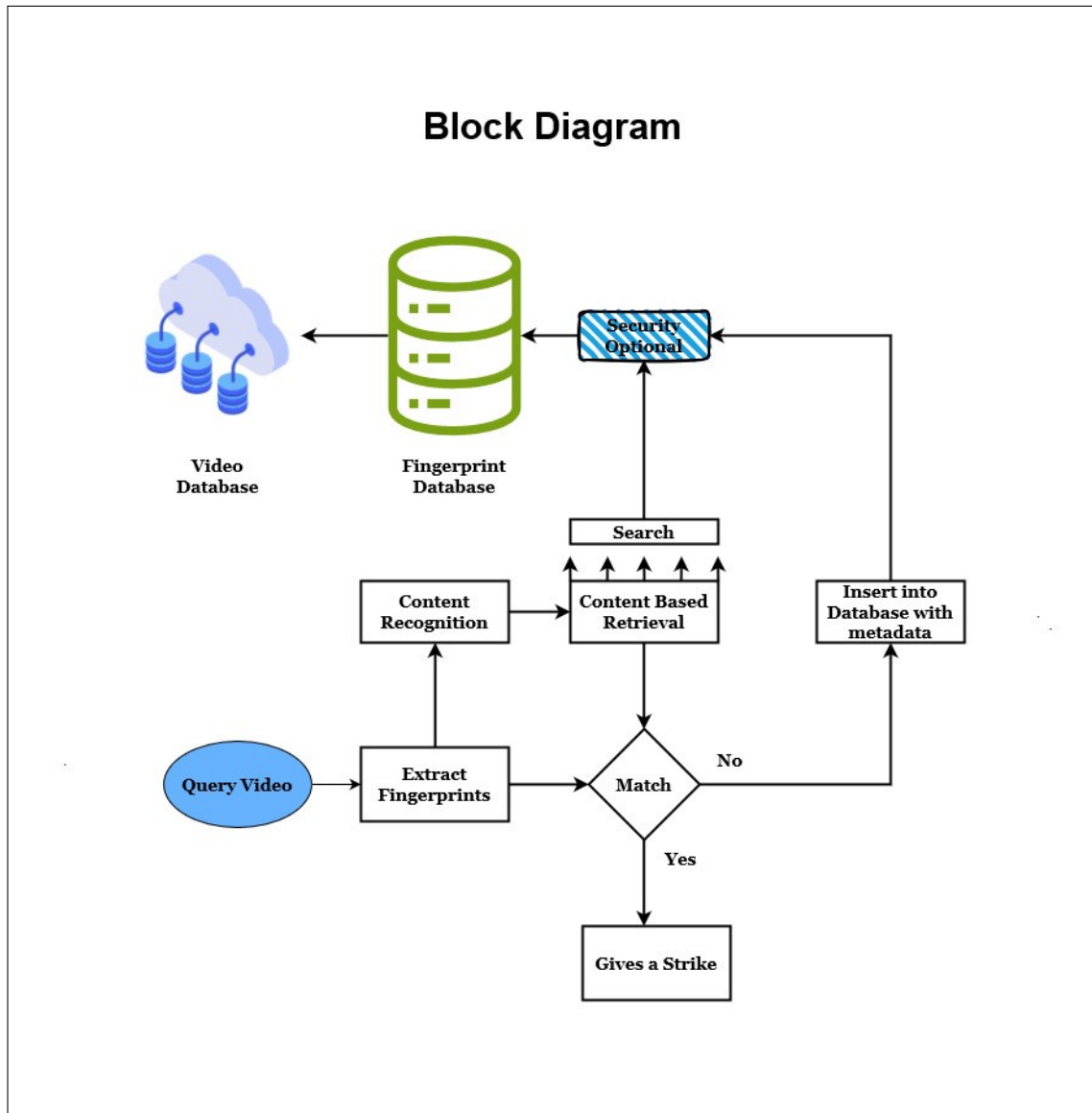


Figure 4.1: Project Architecture

4.2 Data Flow Diagram

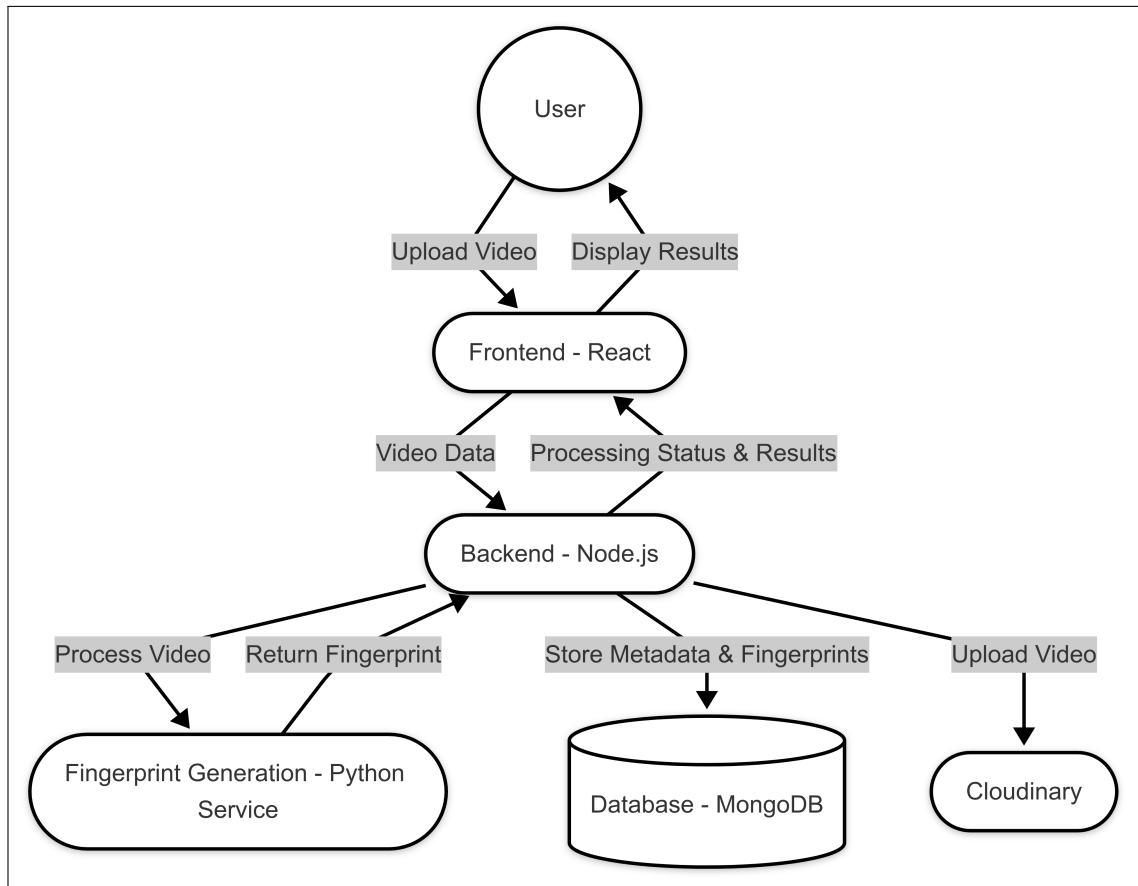


Figure 4.2: Data Flow Diagram

4.3 Class Diagram

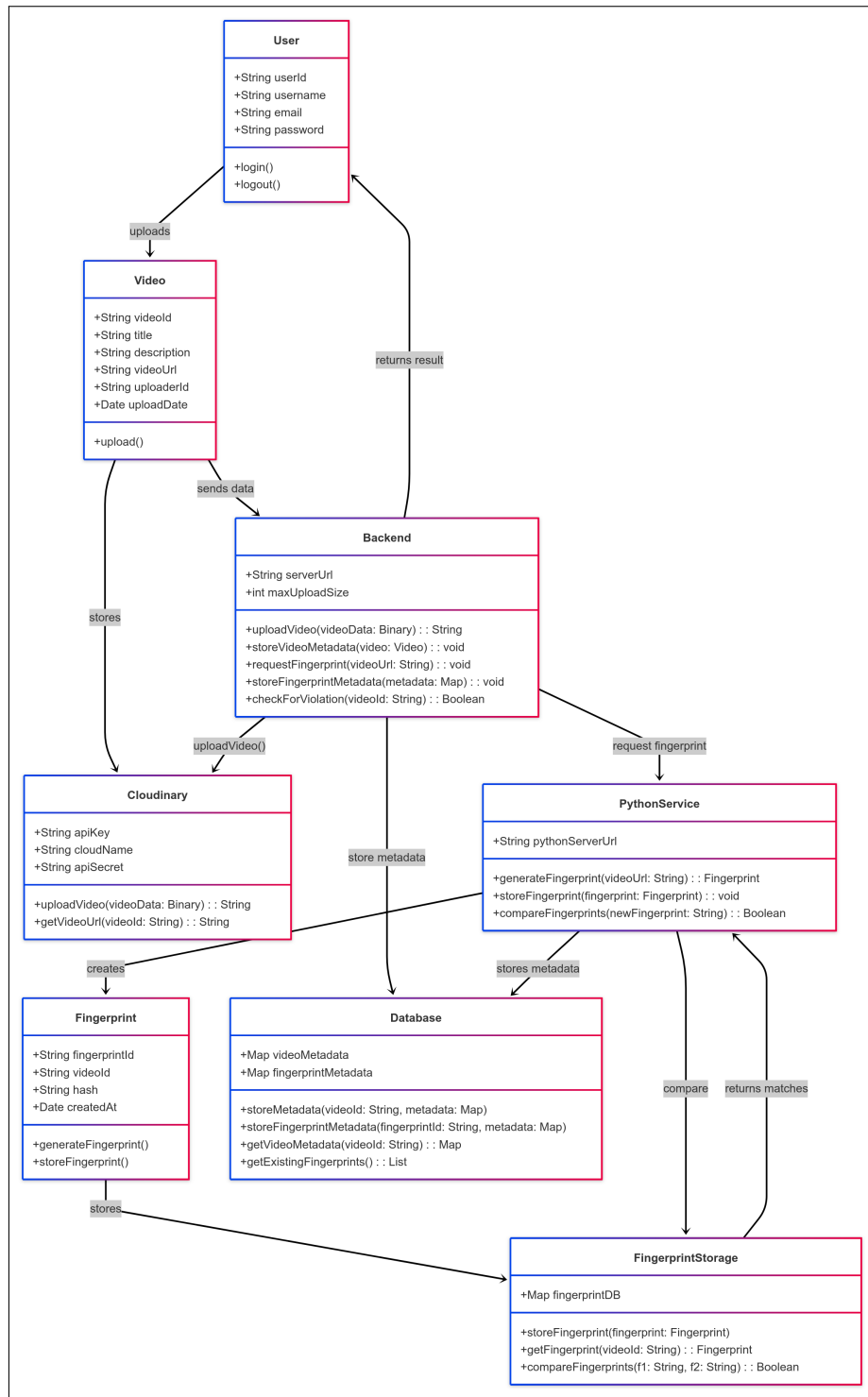


Figure 4.3: Entity - Relationship Diagram

4.4 Use Case Diagram

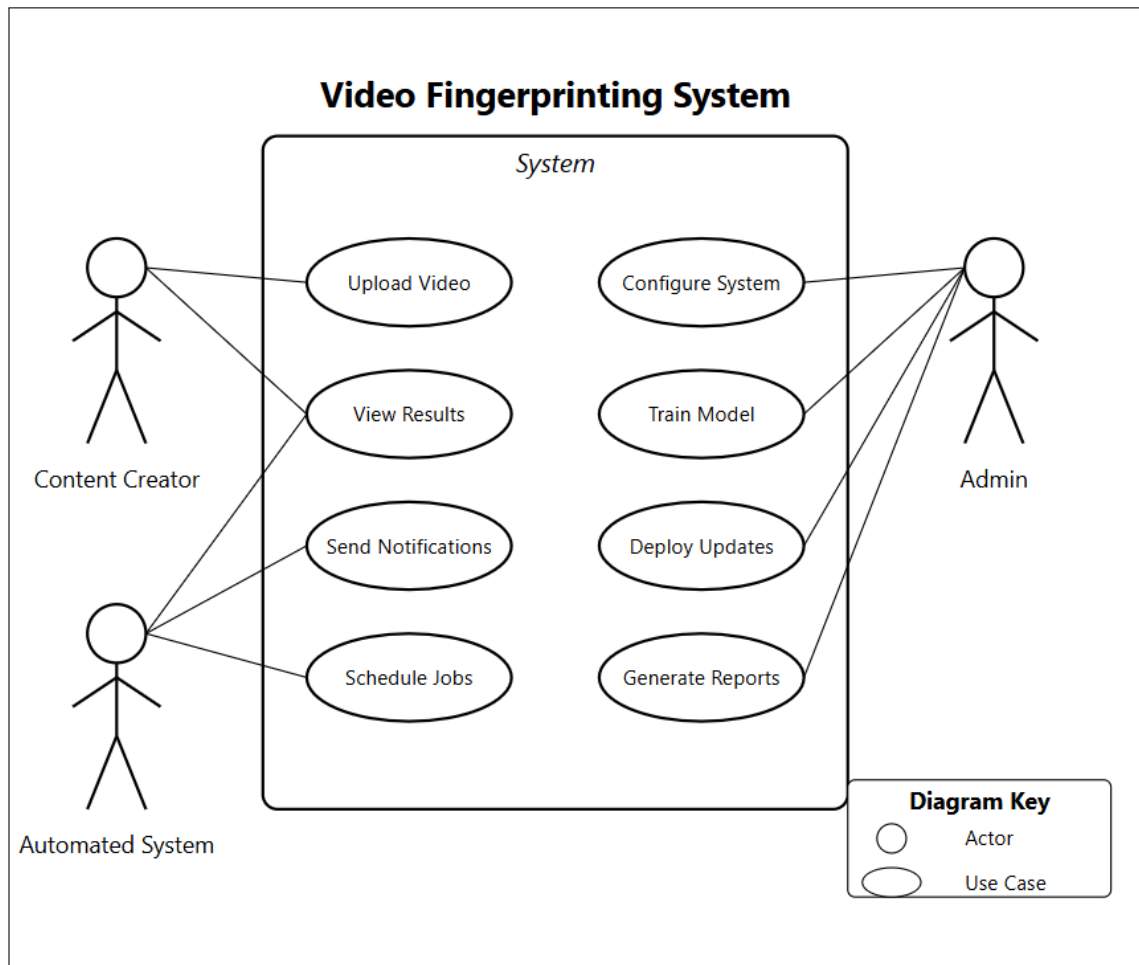


Figure 4.4: Use Case Diagram

4.5 Sequence Diagram

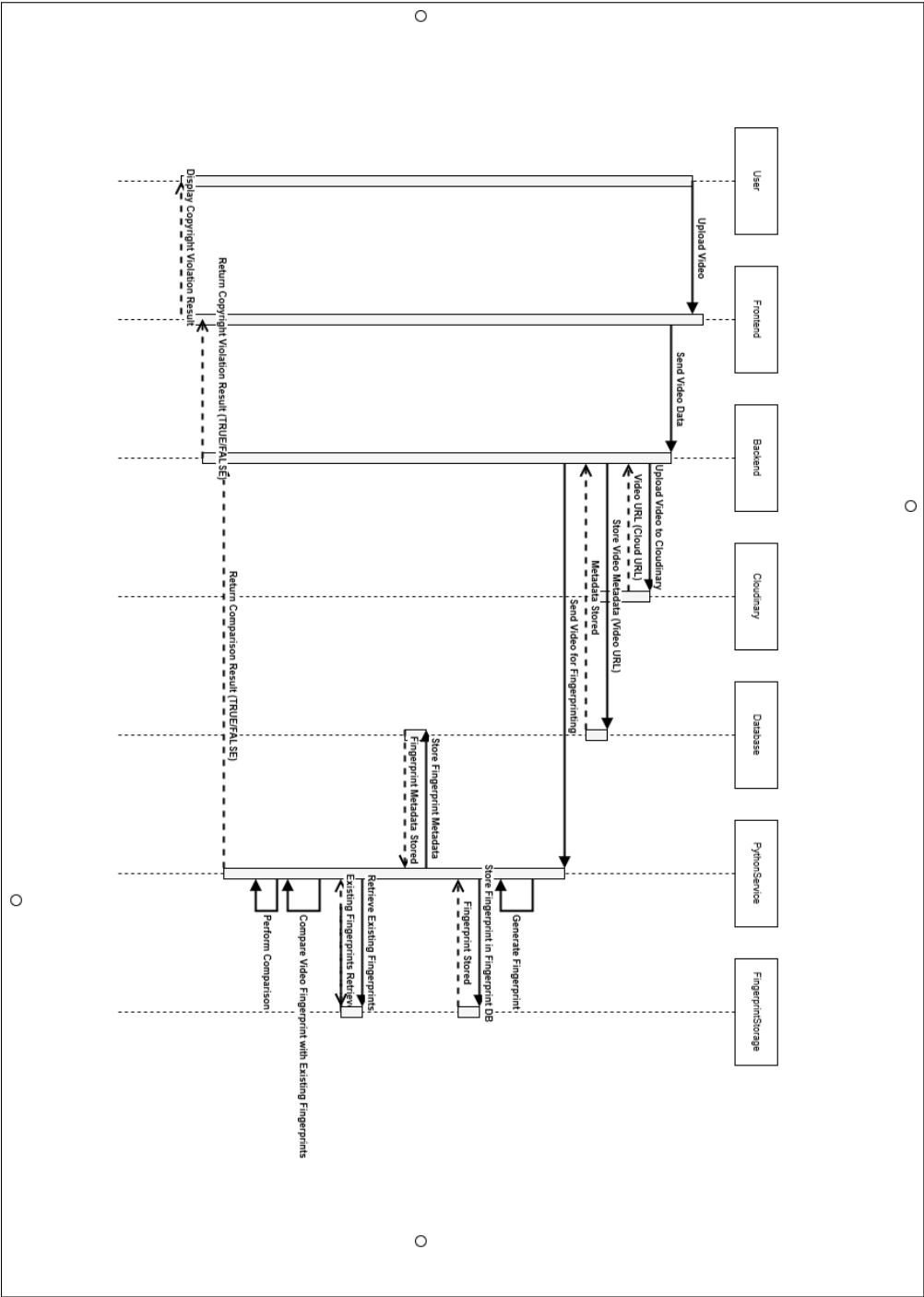


Figure 4.5: Sequence Diagram

CHAPTER 5

PROJECT PLAN

5.1 Project Estimate

5.1.1 Reconciled Estimates

- **Backend Development:** Estimated a total of 4 months (Nov 2024 – Feb 2025)to design and develop the core video processing system, including video fingerprinting, storage management, and copyright detection algorithms. This encompasses setting up cloud infrastructure, implementing video processing pipelines, and developing APIs for front-end integration.
- **Frontend Development:** Estimated 3 (Dec 2024 – Feb 2025) months for developing the user interface, including user dashboards, video upload interfaces, and admin panels. This phase involves implementing responsive designs, user authentication flows, and real-time notification systems.
- **Testing and Security:** Estimated 2(Mar 2025 – Apr 2025) months for comprehensive testing of video processing, fingerprint matching, and security protocols. This includes load testing, penetration testing, and optimization of the copyright detection system.
- **Documentation and Deployment:** Estimated 1 (May 2025) month to provide extensive documentation covering system architecture, API specifications, and deployment guides. This phase includes setting up monitoring systems and establishing operational procedures.

Based on these estimates, the total project timeline would be around 15 months, subject to adjustment based on resource availability and specific requirements.

5.1.2 Project Resources

1. **Backend Developers (3):** Responsible for:
 - Video processing pipeline implementation
 - Fingerprinting algorithm development
 - Database design and optimization
 - Cloud infrastructure setup and management
2. **Frontend Developers (2):** Focus on:

- User interface development
- Dashboard implementation
- Real-time notification system
- Video player integration

3. Project Management Tools:

- GitHub for version control
- Official Group for communication

4. Collaboration Tools:

- Overleaf for report , paper witing
- GitHub for version management .
- MongoDB Atlas for database management

5.2 Risk Management

5.2.1 Risk Identification

- **Technical Risk:**
 1. Performance issues with video processing at scale
 2. False positives in copyright detection
 3. Data storage and bandwidth costs
 4. System security vulnerabilities
- **Financial Risk:** Cost overruns from cloud services and processing requirements
- **Operational Risk:** Scaling challenges with increased user base

5.2.2 Risk Analysis

- **Technical Risk:** High impact, medium likelihood
 - Mitigation: Regular performance testing, optimization sprints
- **Financial Risk:** Medium impact, medium likelihood
 - Mitigation: Usage monitoring, cost optimization strategies
- **Operational Risk:** High impact, low likelihood
 - Mitigation: Scalable architecture, automated monitoring

5.2.3 Overview of Risk Mitigation, Monitoring, Management

- **Technical Risk Mitigation:**
 - Regular code reviews
 - Performance benchmarking
 - Automated testing pipelines
- **Financial Risk Mitigation:**
 - Cloud cost monitoring

- Resource usage optimization
- Regular budget reviews
- **Operational Risk Mitigation:**
 - Automated scaling policies
 - Disaster recovery planning
 - Regular backup procedures
- **Risk Monitoring:**
 - Weekly system performance reviews
 - Monthly security audits
 - Continuous monitoring alerts

5.3 Project Schedule

5.3.1 Project Task Set

- Set up development environments and cloud infrastructure
- Implement basic user authentication and profiles
- Develop video upload and processing pipeline
- Create fingerprinting algorithm and matching system
- Build notification and strike management system
- Implement admin dashboard and moderation tools
- Develop user dashboard and analytics
- Set up CDN and optimize video delivery
- Implement search and categorization features
- Create automated testing suite
- Deploy monitoring and logging systems
- Conduct security audits and penetration testing
- Document APIs and system architecture
- Set up CI/CD pipelines

5.3.2 Timeline Chart

Figure 5.1 is the summarized project timeline chart. It briefly explains the schedule of our entire project from inception to projected completion.

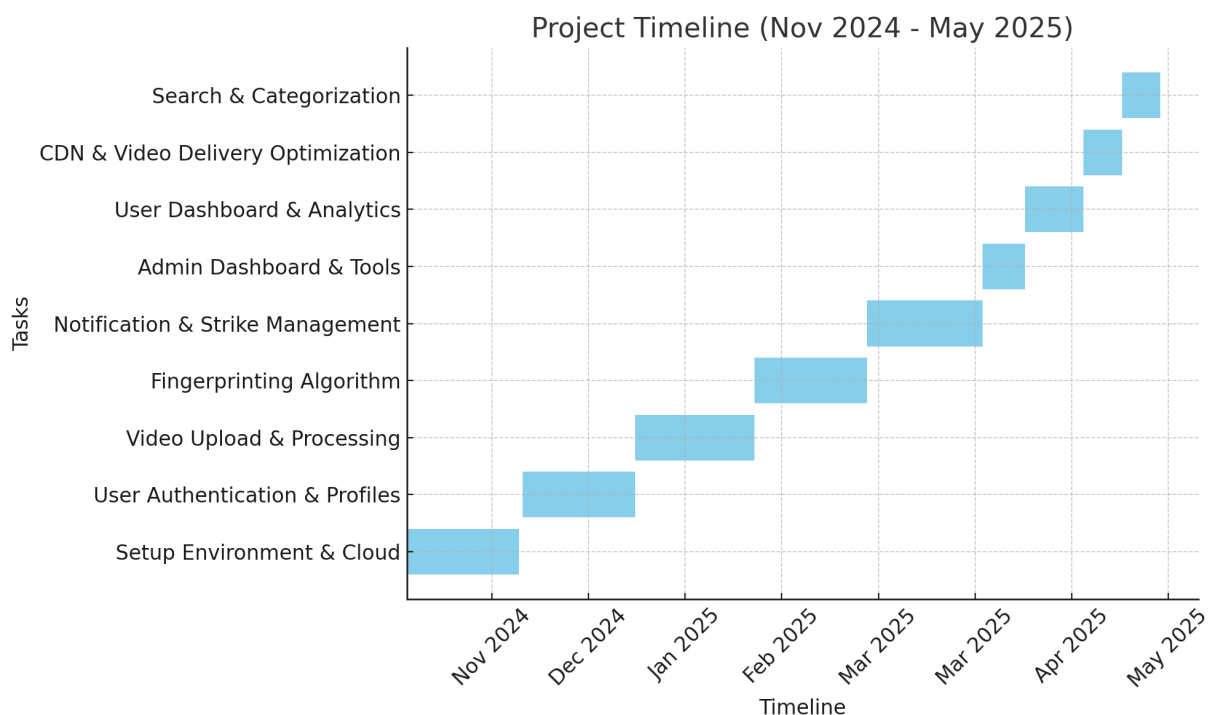


Figure 5.1: Summarized Timeline Chart

5.4 Team Organization

5.4.1 Team Structure

- **Prof . Nikita Kapadnis:** College Mentor, Technical Advisor
- **Ganesh Lokhande:** Frontend Lead, UI/UX Design ,
- **Siddharth Humbe:** Database , API Development ,
- **Piyush Lavekar:** Testing , Application Integration , Report Work
- **Durgesh Choudhary:** Backend Lead, Video Processing ,

CHAPTER 6

PROJECT IMPLEMENTATION

6.1 Stages of Implementation

The development of our project followed a structured pipeline, starting from basic perceptual fingerprinting and progressing toward a robust, modular system for copyright detection. Each stage contributed to refining the fingerprinting, comparison, and decision-making processes.

6.1.1 TIRI-Based Fingerprinting Baseline

We began with the implementation of **Temporally Informative Representative Images (TIRI)** based perceptual hashing. This technique allowed us to generate robust and compact fingerprints from keyframes of videos. The fingerprinting method is designed to be resilient to small transformations like compression artifacts, cropping, and noise.

6.1.2 Modular Python Service

A dedicated Python microservice was built to handle:

- Keyframe extraction
- Fingerprint generation using TIRI-based hashing
- Storage and retrieval of fingerprint data from the MongoDB database
- Comparison of new fingerprints with stored ones

This service communicates with the Node.js backend via REST APIs, making the system flexible and scalable.

6.1.3 Integration with Cloud-Based Storage

To handle media content efficiently, we integrated **Cloudinary** for video storage. Upon upload, the system stores the video on Cloudinary, then extracts keyframes and processes them for fingerprinting. This allowed us to offload media storage and focus on analysis.

6.1.4 Final Pipeline Assembly

The finalized system works as a unified pipeline:

1. User uploads video via the frontend

2. Video is stored on Cloudinary
3. Backend sends video URL to the Python microservice
4. Keyframes are extracted and fingerprinted
5. Generated fingerprints are compared with the database
6. Matching results determine if content is copyrighted

6.1.5 Summary

This modular approach enabled robust detection of duplicate or near-duplicate content. The combination of perceptual fingerprinting and efficient comparison logic ensures high reliability in identifying copyrighted media.

6.2 Project Modules

6.2.1 Overview of Project Modules

1. **Video Upload Module:** Allows users to upload videos through a web interface. Integrates with Cloudinary for storage and provides responsive UI feedback during the upload process.
2. **Fingerprinting Module:** Extracts keyframes and generates TIRI-based fingerprints from videos. Produces compact hash representations that are robust to minor changes.
3. **Metadata Storage Module:** Stores fingerprint hashes, video URLs, and metadata in a MongoDB database for efficient search and comparison.
4. **Comparison & Detection Module:** Compares the generated fingerprints against those stored in the database using a similarity metric (e.g., Hamming distance). Detects potential copyright matches.
5. **Result & Feedback Module:** Returns a binary classification (Copyrighted / Not Copyrighted) along with any matched video data for user reference.

6.2.2 Tools and Technologies Used

- React
- Node.js
- Python
- MongoDB
- Cloudinary
- TensorFlow / OpenCV / NumPy
- Git, GitHub & GitHub Actions

6.2.3 Algorithm Details

Algorithm Flow:

1. User uploads a video through the web application.
2. The video is uploaded to Cloudinary.
3. Backend sends the Cloudinary video URL to the Python microservice.
4. Python service:
 - Extracts keyframes from the video
 - Applies TIRI-based fingerprinting on the keyframes
 - Stores the fingerprints in the MongoDB database
5. On a new upload:
 - Fingerprints are generated for the new video
 - These are compared with existing fingerprints in the database
6. A similarity check is performed using Hamming distance
7. If a match is found above a defined similarity threshold, it is flagged as Copyrighted
8. The final result is sent back to the frontend for user viewing

CHAPTER 7

SOFTWARE TESTING

7.1 Types of Testing

7.1.1 Alpha Testing

Alpha testing was carried out by the development team to check whether the system's core features worked as expected in a controlled environment. The aim was to catch and fix bugs, performance slowdowns, or any incorrect behavior early in development.

Main areas tested during alpha testing:

- Upload functionality through the React frontend
- File handling and API response in the Node.js backend
- Python microservice responsible for generating TIRI-based fingerprints
- MongoDB database storing video data and fingerprints
- The logic for checking duplicates and issuing copyright strikes

Each module was tested separately (unit testing) and then together (integration testing). Scenarios included:

- Uploading the same video more than once
- Uploading edited or low-quality versions of a video
- Testing the system with broken or unsupported video files

7.1.2 Beta Testing

Beta testing was conducted by a selected group of users including students and faculty members, who tested the system in real-life conditions. The goal was to get feedback on ease of use, performance, and the accuracy of copyright detection.

Key aspects examined:

- How user-friendly the web interface was
- How quickly and accurately the system responded
- How well the notification and appeal system worked for flagged videos

Users uploaded various types of videos. The system was able to detect duplicates correctly and provided real-time notifications. Based on feedback, we adjusted similarity thresholds and improved the overall interface experience.

7.2 Integration Testing

7.2.1 Objectives

Integration testing aimed to make sure all parts of the system could communicate and function together as one pipeline. Goals included:

- Making sure uploaded videos move smoothly through all backend processes
- Verifying Cloudinary storage and retrieval works correctly
- Checking that the system returns proper results and handles errors safely

7.2.2 Approach

- **Top-Down Testing:** We began testing from the user interface and moved through backend services and database operations.
- **Mock Services:** Some services like Cloudinary were mocked to isolate internal logic.
- **Automation Tools:** Tools like Postman and test scripts were used to automate API testing and response checking.

Test Case	Description	Expected Outcome
Valid video upload	A standard video file is uploaded through the UI	Video is stored, fingerprint is generated, and no copyright strike
Duplicate video upload	A previously uploaded video is submitted again	The system matches it to an existing fingerprint and flags it
Edited video upload	A slightly changed or compressed version is uploaded	System detects similarity and raises a potential copyright issue
Database disconnection	MongoDB is disconnected during upload	An error is shown, but the system does not crash
Multiple strikes tracking	Repeated uploads of copyrighted videos	Strike count increases, and admin panel updates the record

Table 7.1: Sample Integration Test Cases

7.2.3 Results

Integration testing showed that the different parts of the system worked well together. Some key outcomes included:

- The data flow between modules was reliable and error-free
- Fingerprinting and database operations worked as intended
- Duplicate videos were flagged correctly
- Minor issues were resolved by using logging .

CHAPTER 8

RESULTS

8.1 Output

The implemented video content management system detects copyright infringement through a robust fingerprinting-based detection mechanism. The following steps describe the system workflow and observed outputs during testing and deployment:

System Workflow

1. **Video Upload:** A user uploads a video file via the frontend interface.
2. **Preprocessing:** The system performs frame extraction, resizing, and grayscale normalization to ensure consistent data for hashing.
3. **Fingerprint Generation:** A perceptual hash (pHash) and TIRI-DCT fingerprint are generated for the uploaded video.
4. **Database Matching:** The generated fingerprint is compared against existing fingerprints in the MongoDB database using a similarity threshold.
5. **Infringement Detection:**
 - If a match above the threshold is found, the system flags it as a potential copyright violation.
 - If no significant match is found, the video is accepted and stored.
6. **Action Taken:**
 - A copyright strike is recorded against the uploader.
 - The admin panel displays the flagged video and matching content.
 - The user receives a real-time notification about the strike and is given an option to appeal.

Output Sample

The system was tested using duplicate and original videos. The results are as follows:

- **Input Video:** "user_video.mp4" uploaded.
- **Hash Generated:** 0xAD43E19C...

- **Matching Video Found:** "original_creator_video.mp4"
- **Similarity Score:** 94.6%
- **Action:** Copyright infringement detected.
- **System Response:**
 - Copyright strike issued.
 - Notification sent to user.
 - Video flagged in admin panel for review.

Output

- **User:** user123
- **Strike Count:** 1
- **Video Status:** Blocked due to policy violation
- **Appeal Option:** Enabled

The output confirms that the fingerprint-based detection mechanism effectively identifies duplicates and automates copyright enforcement, offering both precision and speed in video content moderation.

8.2 Screenshots

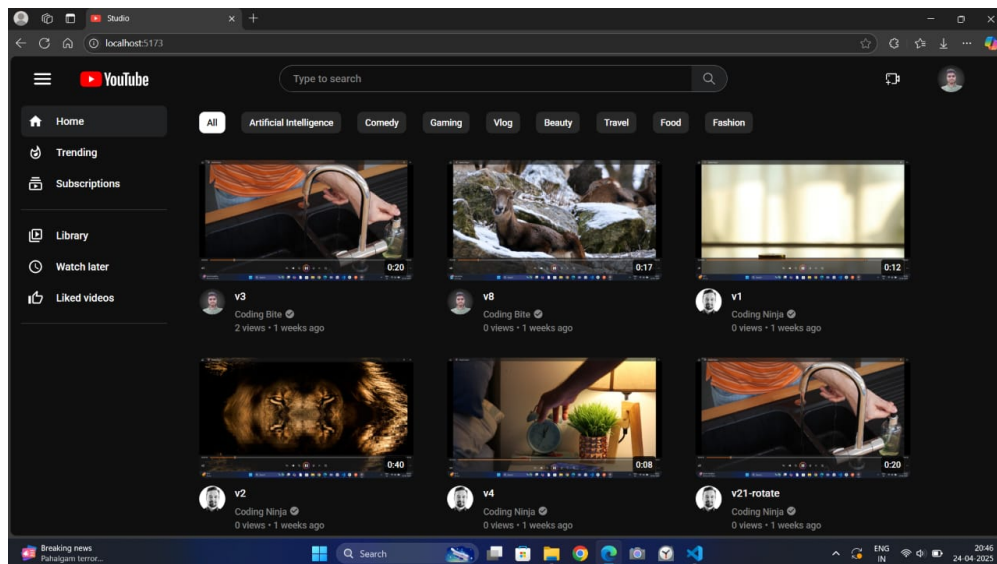


Figure 8.1: Application Interface

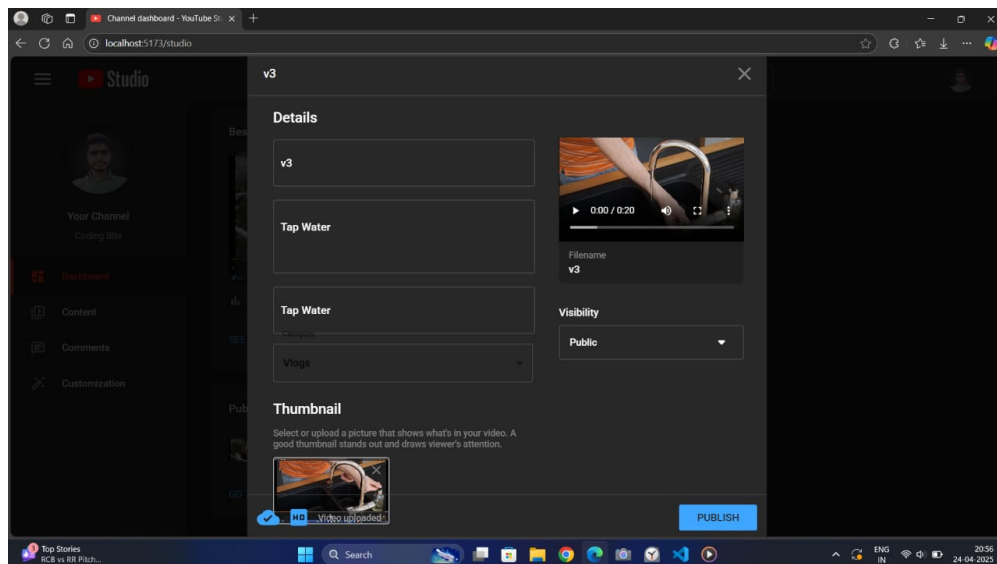


Figure 8.2: Video Uploading stage

Automated Video Content Management System With Copyright Enforcement

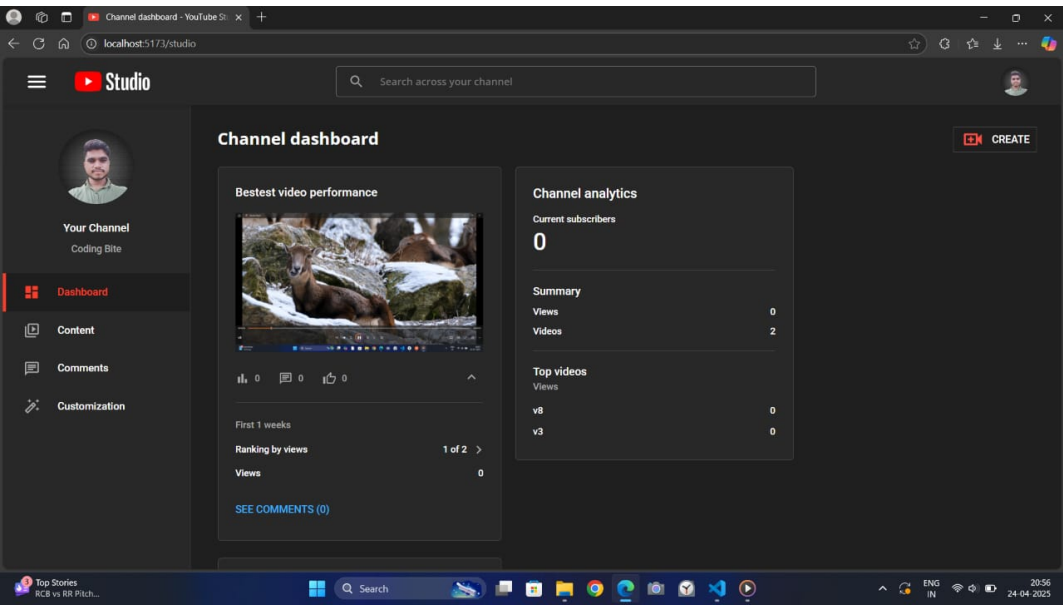


Figure 8.3: Dashboard with uploaded video

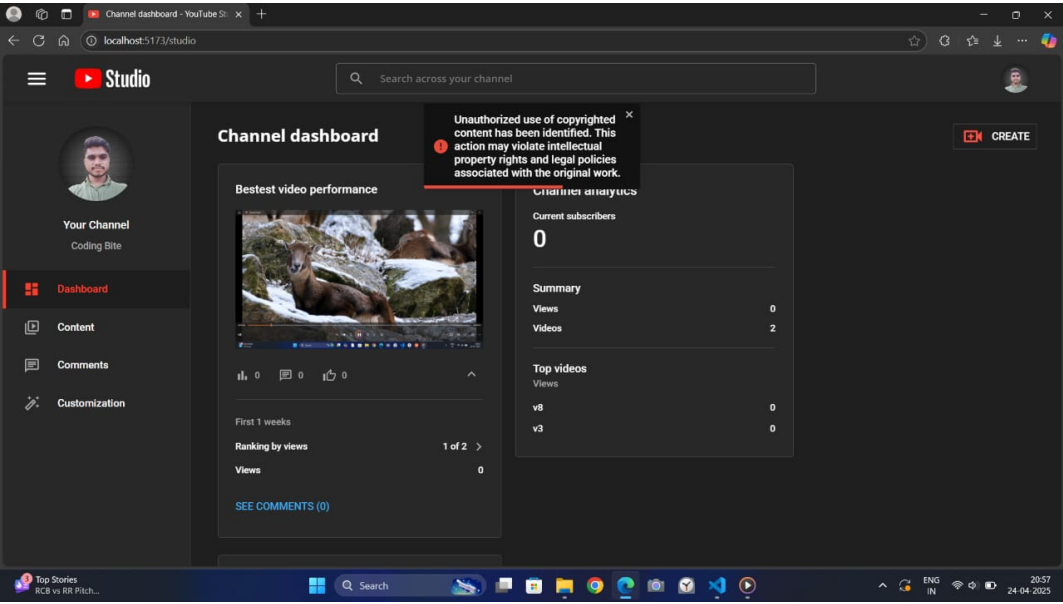


Figure 8.4: Copyright Strike and video removed

Automated Video Content Management System With Copyright Enforcement

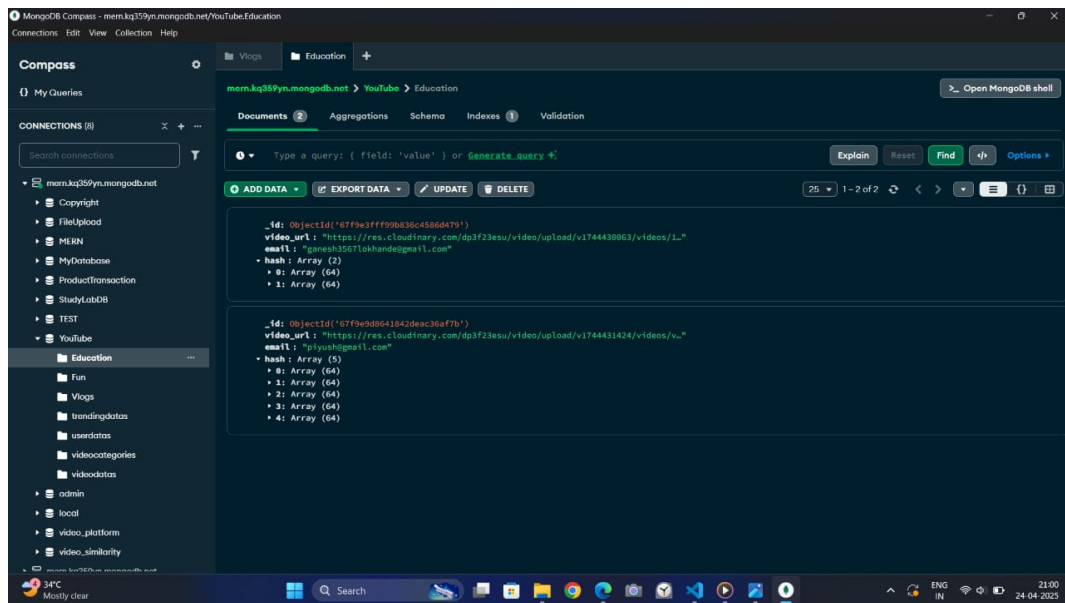


Figure 8.5: backend report

CHAPTER 9

CONCLUSIONS

9.1 Summary

The development of a video-sharing platform with advanced copyright protection marks a major advancement in addressing digital rights management and content authenticity. By integrating video fingerprinting, automated copyright detection, and user-centric interfaces, the platform ensures creators can share content securely while safeguarding their intellectual property.

Strategic architectural design and implementation leverage technologies such as FFmpeg, machine learning for content recognition, and distributed cloud systems to meet the demands of large-scale video processing and content matching. System performance is balanced with user experience through:

- Accurate video fingerprinting with minimal false positives
- Scalable cloud infrastructure for high-volume video uploads
- Intuitive user interfaces for copyright management
- Robust administrative tools for moderation and dispute resolution

Continuous user feedback, performance monitoring, and automated testing will guide ongoing improvements, ensuring responsiveness, security, and reliability. This foundation paves the way for future innovations in secure and fair digital content sharing.

9.2 Applications

The copyright detection system developed in this project can serve a broad range of real-world applications, particularly in domains where digital video content is heavily used and shared. Social media platforms such as YouTube, Instagram, and Facebook are primary candidates for this system. These platforms face continuous issues related to unauthorized re-uploads and copyright violations. By integrating our system, these platforms can proactively detect and restrict the publishing of copied video content, thereby upholding copyright laws and protecting original creators.

Streaming platforms like Netflix, Amazon Prime, and other OTT services can also use this technology to ensure that the content being uploaded by users or partners is not plagiarized. This ensures legal compliance and prevents disputes over intellectual property. Educational platforms such as

Udemy, Coursera, and institutional e-learning portals can benefit from protecting their recorded lectures and tutorial videos from being redistributed without consent.

Individual content creators—such as YouTubers, educators, vloggers, and artists—can use this tool to verify whether their content is being reused without permission. This can help them maintain ownership over their original work and potentially file copyright claims. Furthermore, legal agencies and copyright protection firms can employ this system for automated tracking of pirated video content across the web, reducing manual effort and improving detection efficiency.

9.3 Limitations

Scalability is a concern. As the database of fingerprints grows larger, the computational time and resources required for comparing a new video against existing ones increase significantly. This could lead to performance bottlenecks, especially in a production-level deployment with thousands of videos.

Another limitation is that the current system does not analyze the audio component of videos. If the visual content is significantly altered but the audio remains the same—or if only the audio is copied—our system may not be able to detect the violation. This makes it necessary to eventually incorporate multimodal analysis for complete protection.

9.4 Future Scope

The current system lays a strong foundation for future improvements, and there are several promising areas for enhancement. One of the most impactful additions would be the integration of audio fingerprinting. By analyzing both visual and auditory features of a video, the system would be more robust against edits that affect only one modality, thus increasing detection accuracy.

To address the scalability problem, advanced search and indexing techniques such as Approximate Nearest Neighbor (ANN) algorithms can be implemented. These allow fast and efficient comparisons even with a large dataset of fingerprints, making the system suitable for enterprise-scale applications.

Additionally, the system could benefit from incorporating AI-powered forgery and transformation detection using deep learning. Techniques like

Siamese networks or CNN-RNN hybrids could help detect not only direct copies but also complex transformations and deepfakes.

Lastly, to add a layer of legal credibility and ownership tracking, blockchain technology could be used to store immutable records of video fingerprints. Each video uploaded by a creator could be associated with a unique hash stored on the blockchain, providing verifiable proof of original content ownership in case of legal disputes.

9.5 Conclusion

In this project, we successfully developed a modular system for detecting copyrighted video content using TIRI-based perceptual fingerprinting. The solution allows for reliable identification of duplicate or near-duplicate videos through robust comparison techniques. By integrating with a Node.js back-end, Cloudinary storage, and a MongoDB database, we built a scalable and flexible architecture that can be extended in future work.

Despite some limitations, the system demonstrates strong potential for real-world use in social media, education, and entertainment industries. With the addition of audio analysis, advanced matching algorithms, and blockchain-based ownership tracking, the system can be transformed into a full-fledged copyright protection platform capable of addressing modern digital content challenges.

CHAPTER 10

APPENDIX

10.1 Appendix A

10.1.1 Satisfiability Analysis

The core problem addressed in this project—automated video content management and copyright enforcement—is a combination of video fingerprinting, pattern matching, and copyright policy enforcement tasks. These can be formalized as decision problems, such as: “Is this uploaded video a duplicate of an existing copyrighted work?” or “Does this video match existing fingerprints above a similarity threshold?” These problems are satisfiable if there exists at least one video fingerprint match that satisfies the similarity conditions and the system’s threshold constraints.

In this context, satisfiability is not defined in a Boolean SAT framework but rather over metric spaces involving real-valued similarity scores (e.g., Hamming distance, cosine similarity). Since fingerprint matching involves optimization and threshold-based decision rules, solutions exist where the fingerprint distance is minimized sufficiently, implying the problem is satisfiable under bounded computational resources and data.

10.1.2 Complexity Class Analysis

The tasks in the project can be analyzed from a computational complexity perspective:

- **Video fingerprint generation** (frame extraction, TIRI computation, hashing) occurs in polynomial time relative to the size of the input video frames. Operations like frame resizing, Gaussian filtering, DCT, and hash computation are deterministic and execute in $O(n)$ to $O(n \log n)$ time, placing them in **P (Polynomial Time)**.
- **Fingerprint comparison** using distance metrics like Hamming distance across a database of fingerprints also runs in **polynomial time** with respect to the database size (linear or sub-linear with indexing techniques).
- However, as database size grows exponentially, naïve comparison could simulate aspects of **NP** behavior (e.g., in brute-force nearest-neighbor search), particularly in large-scale deployments. Optimizations like Approximate Nearest Neighbor (ANN) search are necessary to maintain tractable performance.

Thus, while individual operations (hashing, matching) are in $\mathbf{P}$, the overall system's scalability and optimization introduce practical challenges that can approximate NP-Hard behavior without proper engineering.

10.1.3 Modern Algebra Perspective

From an algebraic perspective, the video fingerprinting and matching tasks can be modeled as functions and operations on structured sets:

- Videos are transformed into vectors (fingerprints) over discrete spaces (e.g., binary strings representing hashes).
- Fingerprinting functions can be represented as mappings $f : V \rightarrow H$, where V is the space of videos and H is the space of fingerprints.
- Similarity comparison operations define a **metric space** (H, d) , where d is the distance function (e.g., Hamming distance) satisfying properties like non-negativity, identity, symmetry, and the triangle inequality.
- The act of comparing videos through their fingerprints can be understood as finding equivalence classes in H , where two videos belong to the same class if their distance is below a defined threshold.

Moreover, copyright enforcement (strike management) can be abstractly viewed as a monoid action, where a set of user states (number of strikes) is modified under operations (strike increments) according to system rules.

Thus, using modern algebraic structures like metric spaces, monoids, and function mappings, the fingerprinting and copyright enforcement mechanisms are formally definable and mathematically tractable.

10.2 Appendix B

10.2.1 Survey Paper Details

- **Paper Status:** Published
- **Name of the Conference:** 2024 IEEE 16th International Conference on Computational Intelligence and Communication Networks (CICN)
- **Title:** Exploring New Frontiers in Video Fingerprinting: A Comprehensive Review of Techniques
- **DOI:** 10.1109/CICN63059.2024.10847472
- **Date Added to IEEE Xplore:** 27 January 2025



Figure 10.1: IEEE Xplore Snapshot

Authors
Siddharth Humbe Department of Computer Engineering, Pune Institute of Computer Technology, Pune, India
Piyush Lavekar Department of Computer Engineering, Pune Institute of Computer Technology, Pune, India
Nikita Kapadnis Department of Computer Engineering, Pune Institute of Computer Technology, Pune, India
Durgesh Choudhary Department of Computer Engineering, Pune Institute of Computer Technology, Pune, India
Ganesh Lokhande Department of Computer Engineering, Pune Institute of Computer Technology, Pune, India

Figure 10.2: Authors

10.2.2 Implementation Paper Details

- **Paper Status:** Applying
- **Name of the Conference Proposed :** IEEE Madhya Pradesh Section Conference 2025

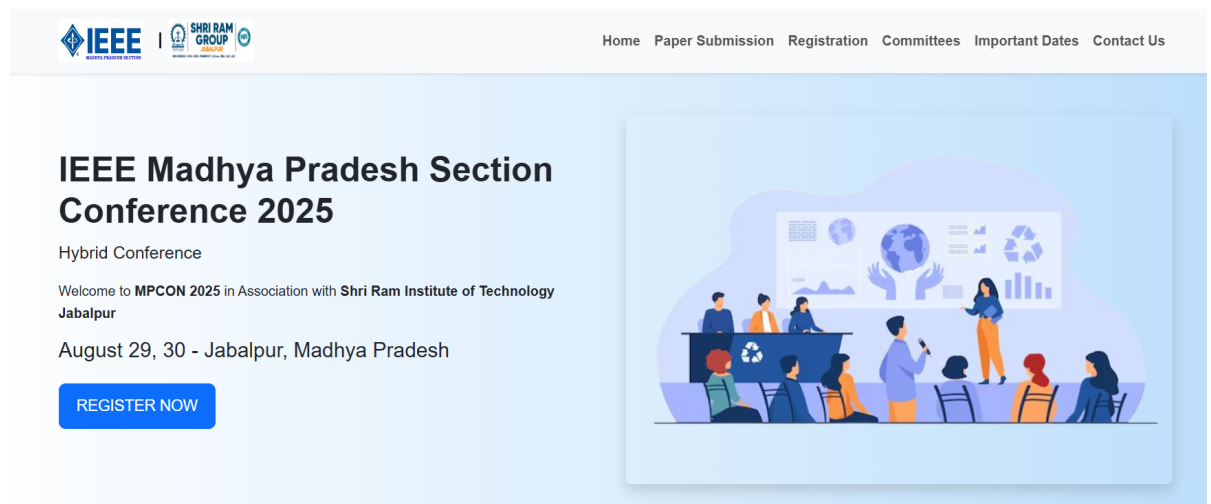


Figure 10.3: Conference Details

10.3 Appendix C

10.3.1 Plagiarism Report

The plagiarism report for the project report has been attached herewith.

Similarity: 6 %

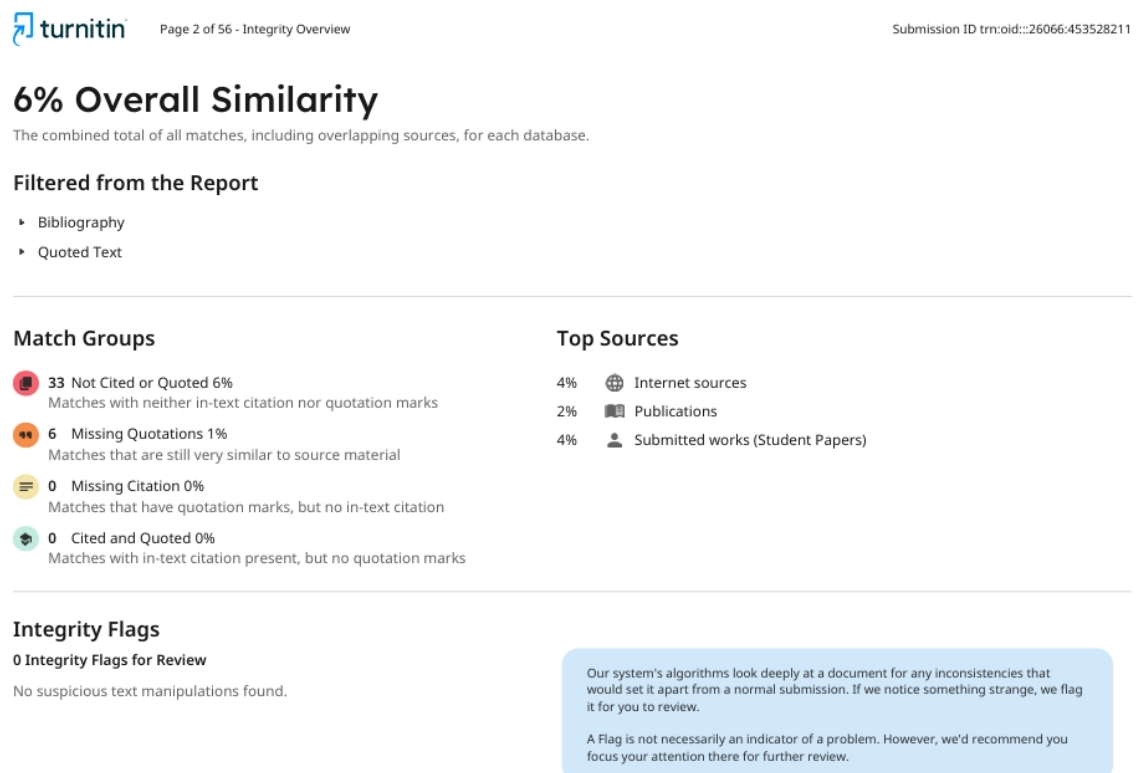


Figure 10.4: Overall Similarity Report

Durgesh Choudhary

RE-2022-557358

-  Batch 6
-  Batch 6
-  Universidad del Valle

Document Details

Submission ID
trn:oid:::26066:453528211

Submission Date
Apr 28, 2025, 8:30 PM GMT+5:30

Download Date
Apr 28, 2025, 8:35 PM GMT+5:30

File Name
RE-2022-557358.pdf

File Size
1.8 MB

51 Pages

6,113 Words

38,038 Characters

Figure 10.5: Document Details

CHAPTER 11

REFERENCES

- [1] Khelifi, F., & Bouridane, A. (2019). "A Perceptual Video Hashing System for Content Authentication and Identification." *IEEE Transactions on Circuits and Systems for Video Technology*, 29(4), 1049–1062. <https://doi.org/10.1109/TCSVT.2017.2773499>
- [2] Kim, M., Park, Y., & Kim, H. (2023). "Adversarial Compression-Aware Training for Robust Deep Video Classification." *IEEE Transactions on Pattern Analysis and Machine Intelligence*. <https://doi.org/10.1109/TPAMI.2023.3240307>
- [3] Esmaeili, M., & Ward, R. (2010). "Robust Video Hashing Using TIRI-Based Features." *IEEE Transactions on Information Forensics and Security*, 5(4), 675–681. <https://doi.org/10.1109/TIFS.2010.2072960>
- [4] Esmaeili, M., Ward, R., & Fatourehchi, M. (2011). "A Secure and Robust Video Copy Detection System Using TIRI-Based Features." *Proceedings of IEEE ICASSP*, 2296–2299. <https://doi.org/10.1109/ICASSP.2011.5946911>
- [5] Esmaeili, M., & Ward, R. (2010). "Robust and Fast Video Copy Detection Using TIRI-DCT Features." *IEEE Transactions on Circuits and Systems for Video Technology*, 21(12), 1847–1857. <https://doi.org/10.1109/TCSVT.2011.2168277>
- [6] Chao, Y. (2008). "Video Fingerprinting and Similarity Detection." [Online]. Available: <http://www.ifp.uiuc.edu/yunchao2/report.pdf>
- [7] Sarkar, A., Ghosh, P., Moxley, E., & Manjunath, B. S. (2008). "Video Fingerprinting: Features for Duplicate and Similar Video Detection and Query-based Video Retrieval." *Proceedings of SPIE Media Forensics and Security*.
- [8] Aguilera, H., García, R., Prieto-Castrillo, F., & Rodríguez, A. (2021). "Video Tampering Detection Based on Optical Flow and Semantic Segmentation." *Sensors*, 21(6), 2059. <https://doi.org/10.3390/s21062059>
- [9] Singh, R., Anand, R. S., Singh, M., Singh, D., & Kumar, A. (2020). "Content Based Video Retrieval Systems—Review, Trends and Challenges." *Journal of Visual Communication and Image Representation*, 71, 102773. <https://doi.org/10.1016/j.jvcir.2020.102773>
- [10] Liu, H., Lu, H., Wen, Z., & Xue, X. (2012). "Gradient Ordinal Signature and Fixed-Point Embedding for Efficient Near-Duplicate Video Detection." *IEEE Transactions on Circuits and Systems for Video Technology*, 22(4), 555–568. <https://doi.org/10.1109/TCSVT.2011.2168294>